



Model-Free Control

Blink Sakulkueakulsuk

Lecture 1

Agent

Action

Environment

Reward

Next State



Agent

Action

Exploration &
Exploitation

Environment

Reward

Next State



Agent

Value Functions

Action

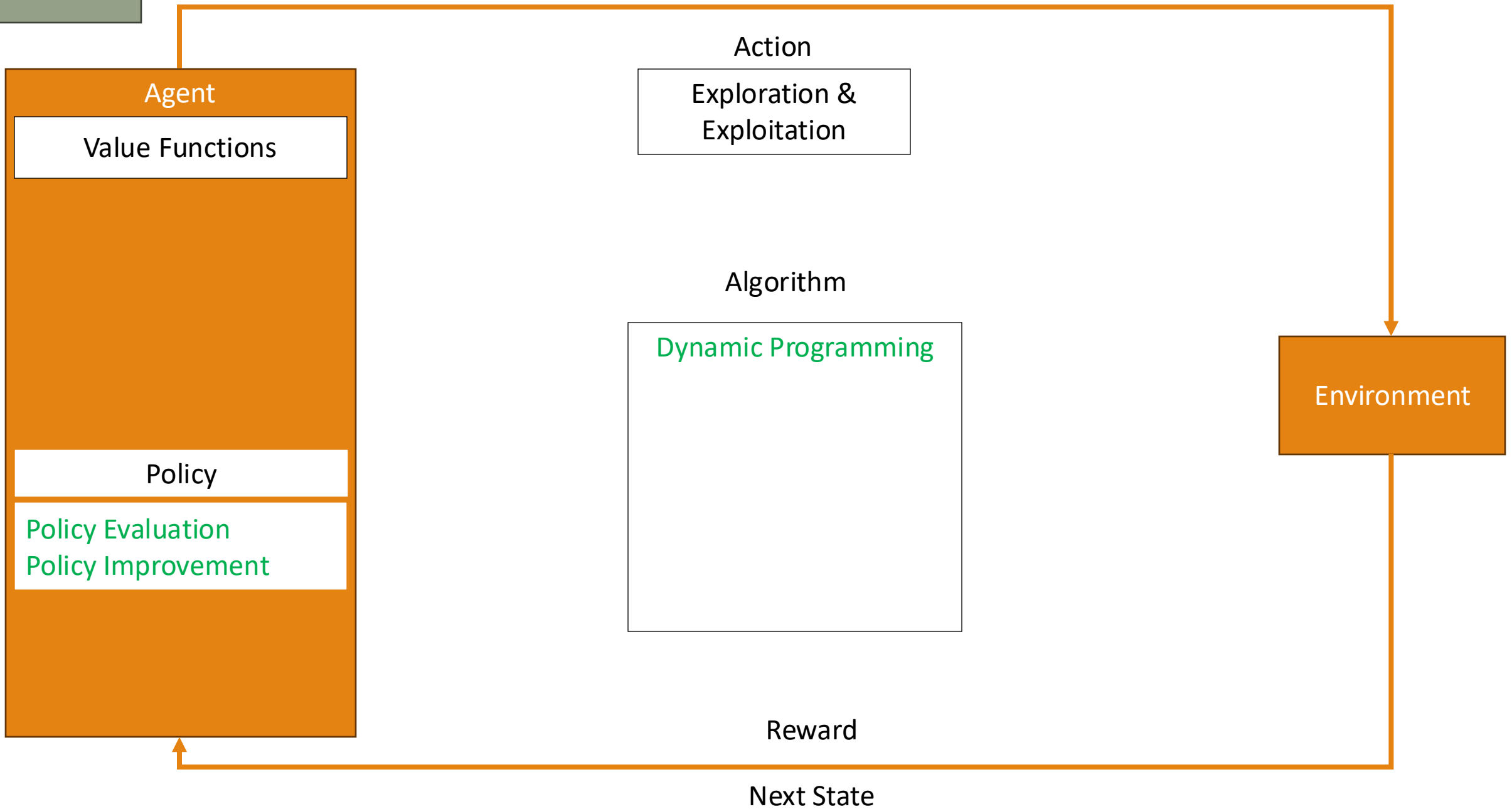
Exploration &
Exploitation

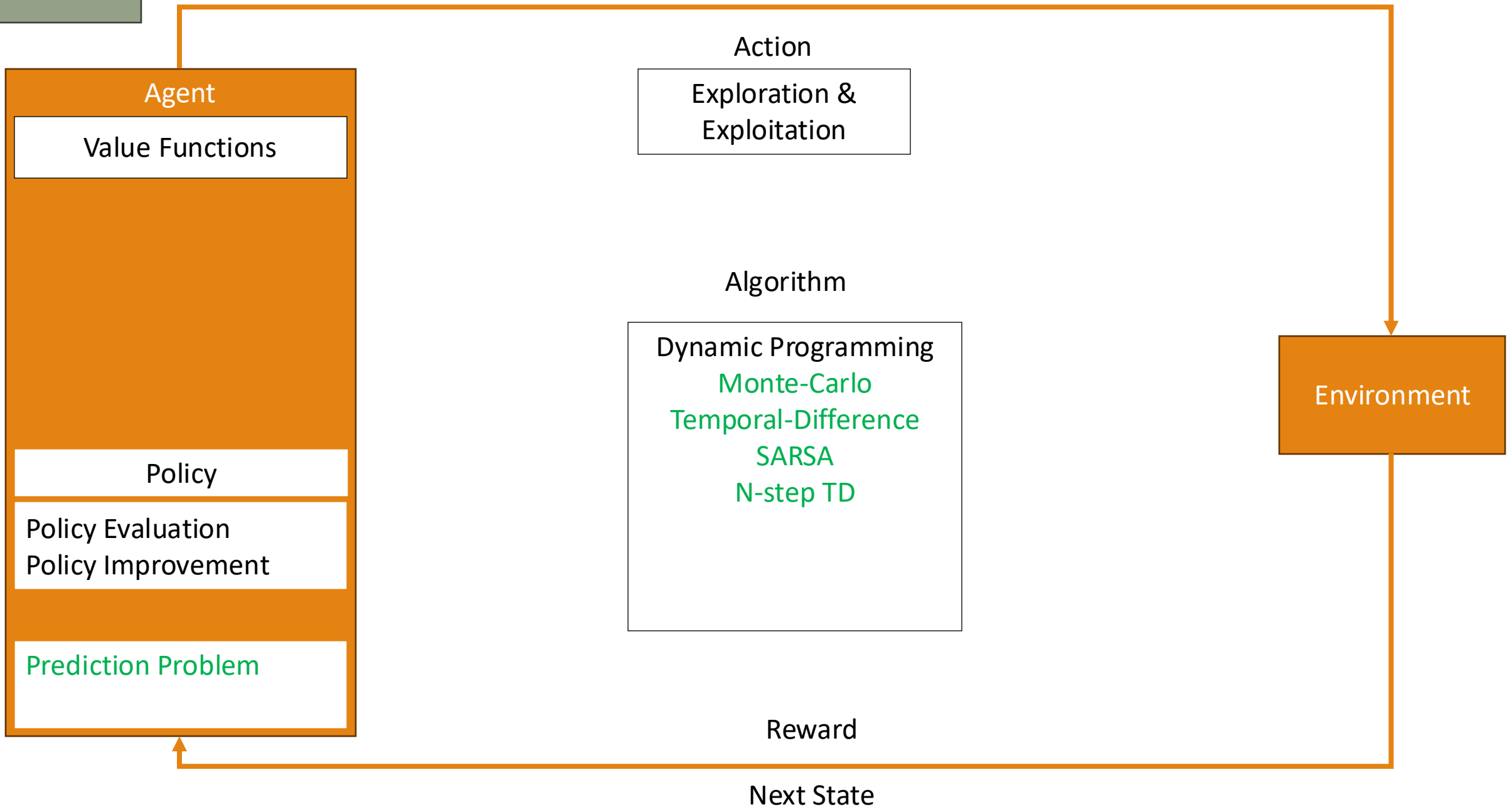
Environment

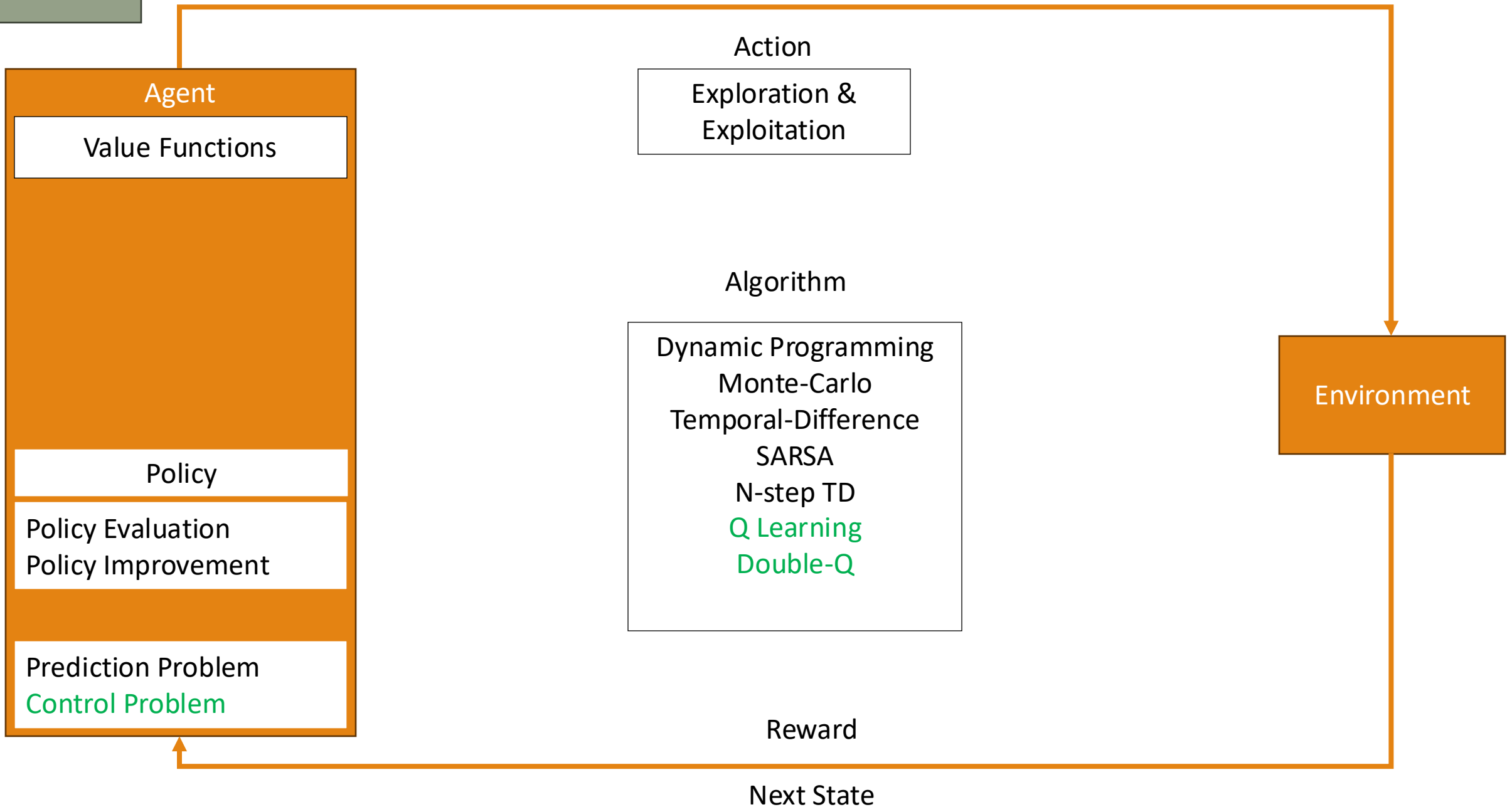
Reward

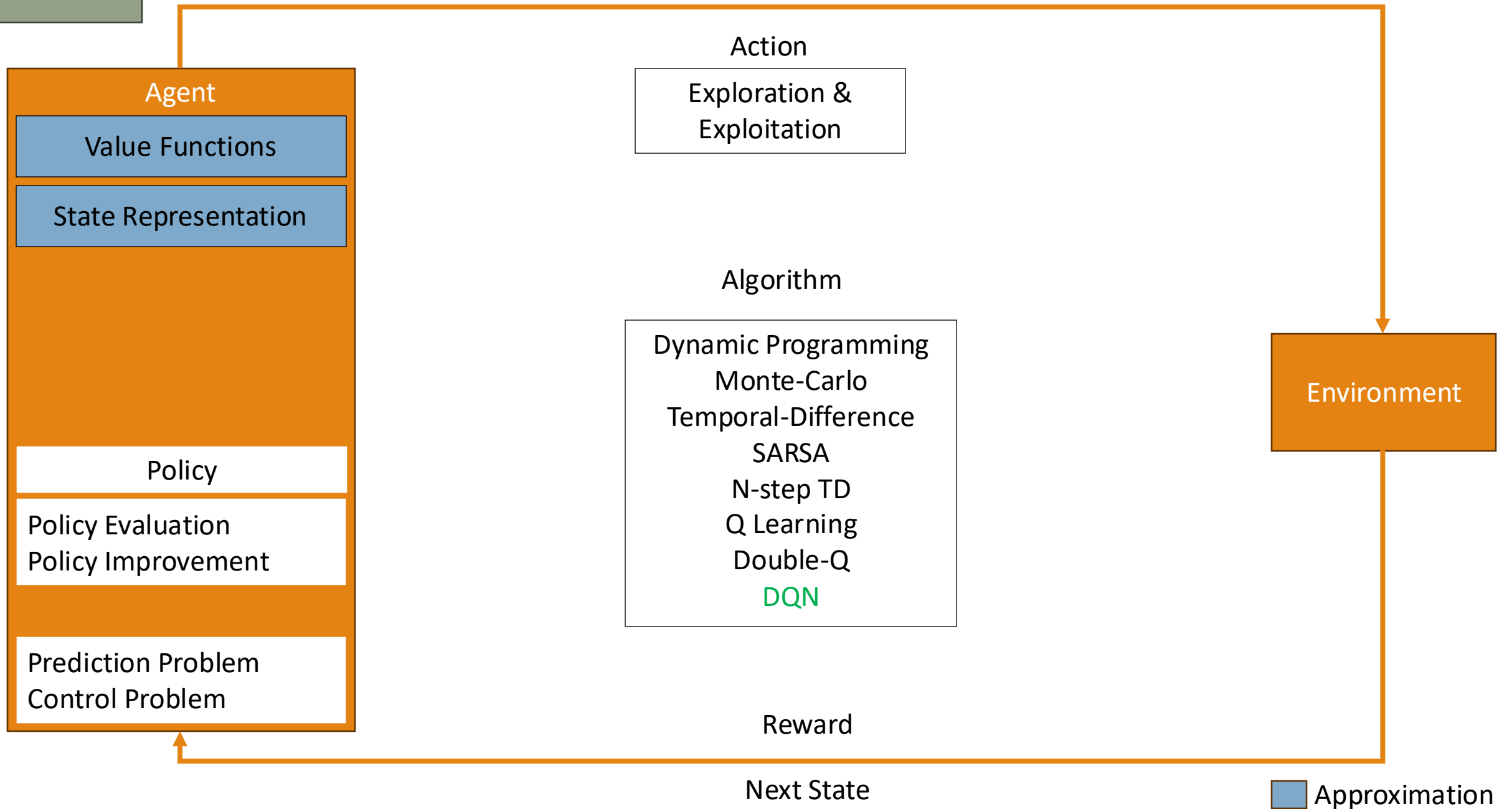
Next State











Four main Bellman Equations

Bellman Expectation Equation

$$v_{\pi}(s) = \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s, A_t \sim \pi(s)]$$

$$q_{\pi}(s, a) = \mathbb{E}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]$$

Bellman Optimality Equation

$$v^*(s) = \max_a \mathbb{E}[R_{t+1} + \gamma v^*(S_{t+1}) | S_t = s, A_t = a]$$

$$q^*(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_{a'} q^*(S_{t+1}, a') \mid S_t = s, A_t = a \right]$$

Prediction and Control

Prediction

- A problem when we perform **policy evaluation**
- Estimating v_π or q_π

Control

- A problem when we perform **policy optimization**
- Estimating v^* or q^*

Evaluating Policy

$$\pi \geq \pi' \leftrightarrow v_{\pi}(s) \geq v_{\pi'}(s) , \forall s$$

One policy is better than or equal to another **if and only** if its value function is greater or equal to another.

Policy Evaluation

Given a policy, we want to estimate this

$$v_{\pi}(s) = \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | s, \pi]$$

We initialize all value to zero, and we iterate the equation above as an update

$$v_{k+1}(s) = \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) | s, \pi] , \quad \forall s$$

Note that

$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_r \sum_{s'} p(r, s' | s, a) (r + \gamma v_{\pi}(s'))$$

If $v_{k+1}(s) = v_k(s), \forall s$, we solve v_{π} .

Policy Improvement

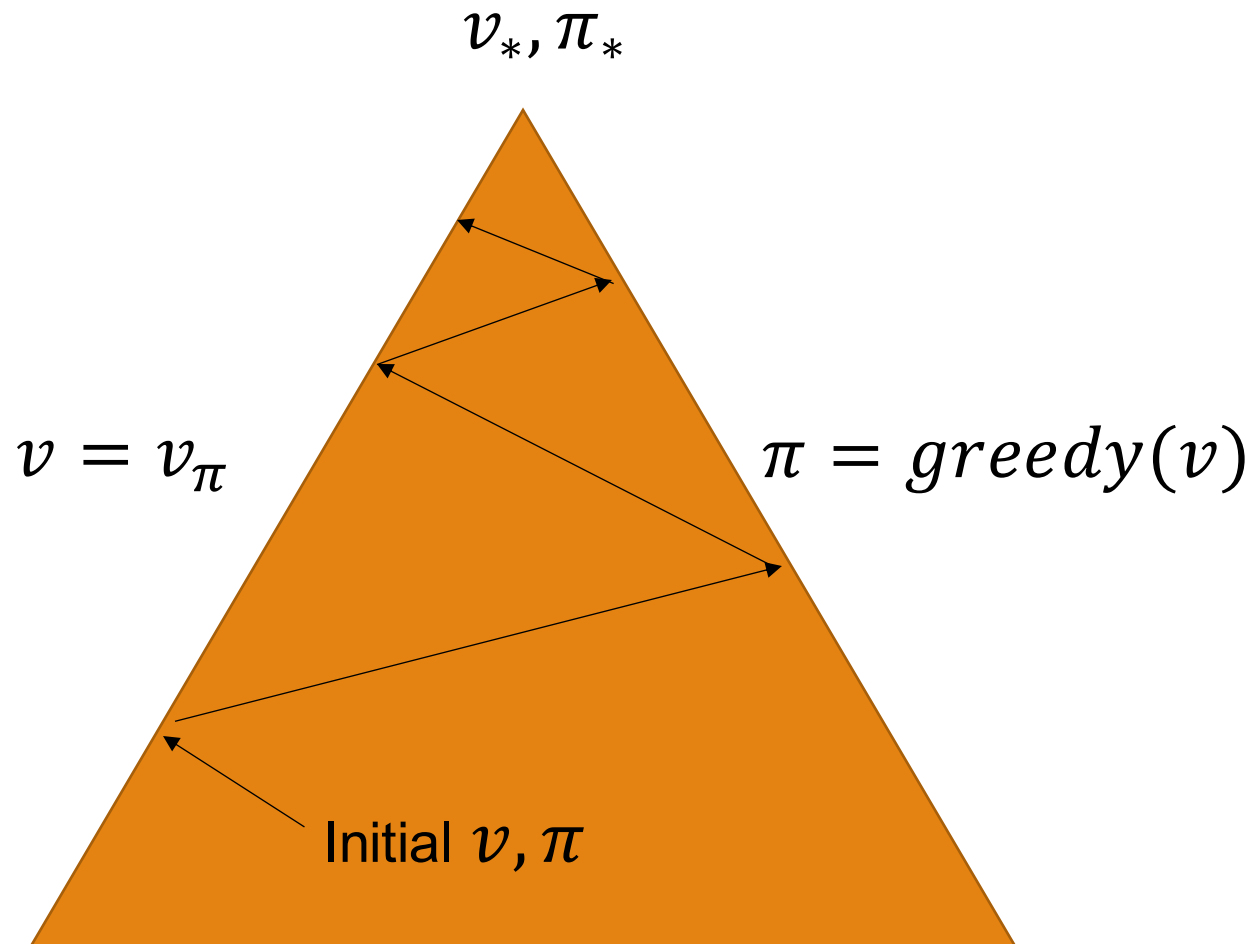
The example shows that we can improve a policy when we learn the value.

- In the example, we do not improve any policy

$$\begin{aligned}\forall s: \pi_{new}(s) &= \operatorname{argmax}_a q_{\pi}(s, a) \\ &= \operatorname{argmax}_a \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s, A_t = a]\end{aligned}$$

We use the new policy to evaluate, and repeat.

Policy Iteration



Starting with initial v, π

Loop do:

Perform policy evaluation,

$$v = v_\pi$$

Perform policy improvement,

$$\pi' \geq \pi$$

Value Iteration

Or we can learn the policy on the fly

- We update the policy every iteration

We can take the Bellman optimality equation as an update

$$v_{k+1}(s) \leftarrow \max_a \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) | S_t = s, A_t = a] \quad , \quad \forall s$$

It is the same as **policy iteration** with $k = 1$. Basically, we improve the policy every iteration.

Dynamic Model

$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_r \sum_{s'} p(r, s' | s, a) (r + \gamma v_{\pi}(s'))$$

The value function above contains the dynamic of the problem

- $p(r, s' | s, a)$ tells the transition model of the problem. This is essentially the dynamic of the problem.

Key Question: What if we do not know that? Can the agent still learn?

- Spoiler Alerted: Yes, it can. (Otherwise, we wouldn't have this class)

Monte-Carlo Policy Evaluation

Given the sequential decision problem, MC learns v_π from episodes under policy π

$$S_1, A_1, R_2, \dots, S_k \sim \pi$$

The return is calculated given an ending time T

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t-1} R_T$$

The value function is then calculated from the expected return

$$v_\pi(s) = \mathbb{E}[G_t | S_t = s, \pi]$$

We can use **sample average return** instead of **expected return**

- This is called **Monte Carlo policy evaluation**

Value Function Approximation

In dynamic programming with MDP, we use **lookup tables** to learn value function.

- A mapping of state s to a value $v(s)$
- A mapping of state-action pair s, a to a state-action value $q(s, a)$

But we cannot do that for a large MDP

- Infeasible to store all information due to the **curse of dimensionality**
- **Too slow to learn** the value of each state **individually**
- States are often **not fully observable**

Function Approximation Example

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha \mathbb{E}[(R_{t+1} - q_{\mathbf{w}}(S_t, A_t)) \nabla_{\mathbf{w}_t} q_{\mathbf{w}}(S_t, A_t)]$$

If we use a linear function approximation,

$$q(s, a) = \mathbf{w}^T \mathbf{x}(s, a)$$

The gradient becomes

$$\nabla_{\mathbf{w}_t} q_{\mathbf{w}}(S_t, A_t) = \mathbf{x}(s, a)$$

Then the SGD update is

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha (R_{t+1} - q_{\mathbf{w}}(S_t, A_t)) \mathbf{x}(s, a)$$

Linear Update = step-size x prediction error x feature vector

Non-linear Update = step-size x prediction error x gradient

Temporal Difference Learning

$$v_{k+1}(s) = \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) | S_t = s, A_t \sim \pi(S_t)]$$

Instead of calculating the expected return, we can sample the value.

$$v_{t+1}(S_t) = R_{t+1} + \gamma v_t(S_{t+1})$$

But instead of updating everything on the noisy value, we can update the value a little bit instead.

$$v_{t+1}(S_t) = v_t(S_t) + \alpha_t(R_{t+1} + \gamma v_t(S_{t+1}) - v_t(S_t))$$

MC Prediction vs TD Prediction

Prediction problem: learn v_π online from experience under policy π

Monte Carlo:

- Update value $v_n(S_t)$ with respect to sample return G_t

$$v_n(S_t) = v_n(S_t) + \alpha(G_t - v_n(S_t))$$

Temporal-difference learning:

- Update value $v_t(S_t)$ with respect to estimated return $R_{t+1} + \gamma v(S_{t+1})$

$$v_{t+1}(S_t) = v_t(S_t) + \alpha_t(R_{t+1} + \gamma v_t(S_{t+1}) - v_t(S_t))$$

Bootstrapping and Sampling

Bootstrapping

- Use the estimate of the next state to update
- DP and TD use
- MC does not

Sampling

- Update samples an expectation
- MC and TD sample
- DP does not

SARSA

We can also learn action values by updating value $q_t(S_t, A_t)$ with respect to estimated return $R_{t+1} + \gamma v(S_{t+1}, A_{t+1})$

$$q_{t+1}(S_t, A_t) = q_t(S_t, A_t) + \alpha_t(R_{t+1} + \gamma q_t(S_{t+1}, A_{t+1}) - q_t(S_t, A_t))$$

This algorithm is called SARSA, because it uses $S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}$

Multi-Step Returns

In general, we can formalize n-step return as

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} v(S_{t+n})$$

We can define multi-step temporal-difference learning as

$$v(S_t) \leftarrow v(S_t) + \alpha(G_t^{(n)} - v(S_t))$$

Mixing Multi-Step Returns

Consider a multi-step return equation,

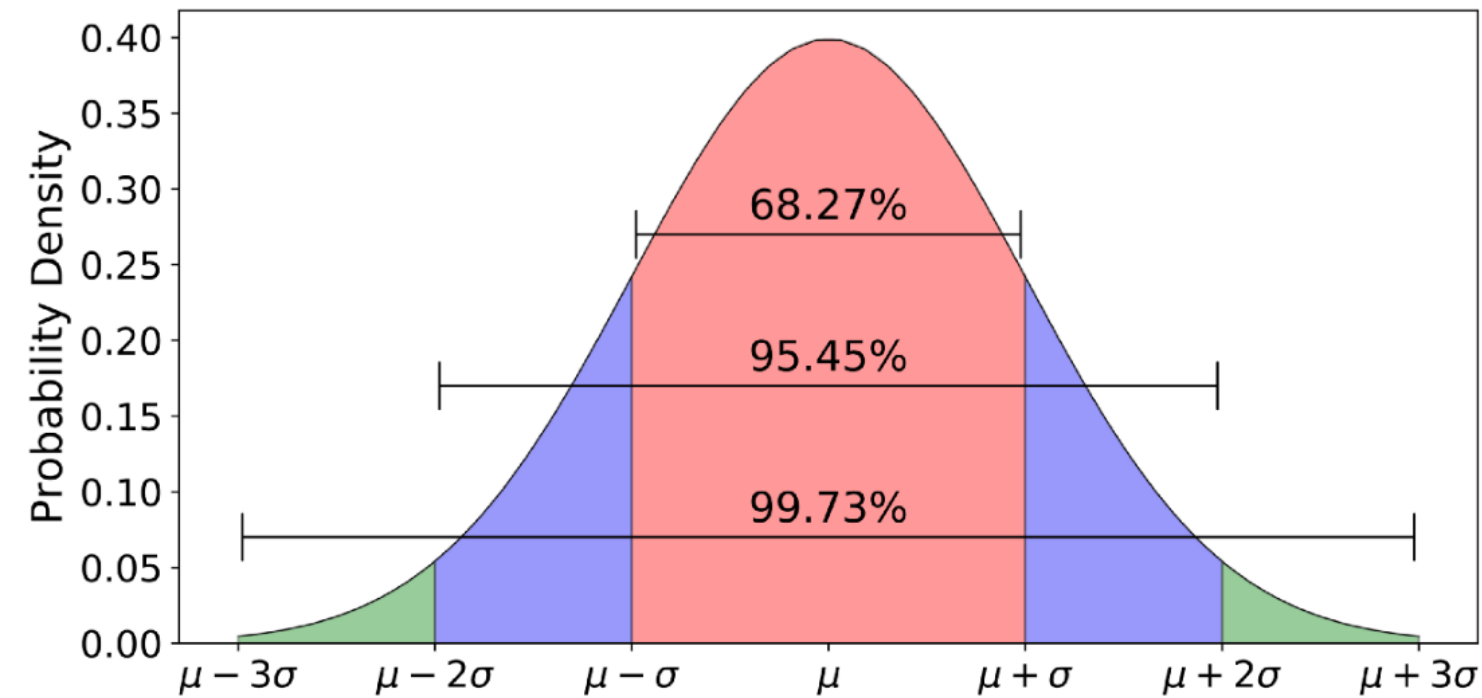
$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} v(S_{t+n})$$

Multi-step returns bootstrap on one state, $v(S_{t+n})$. However, we can modify the equation so you can bootstrap a little bit on many states

$$G_t^\lambda = R_{t+1} + \gamma((1 - \lambda)v(S_{t+1}) + \lambda G_{t+1}^\lambda)$$

Understanding Distributions

Understanding Distributions



What is x axis?

What is y axis?

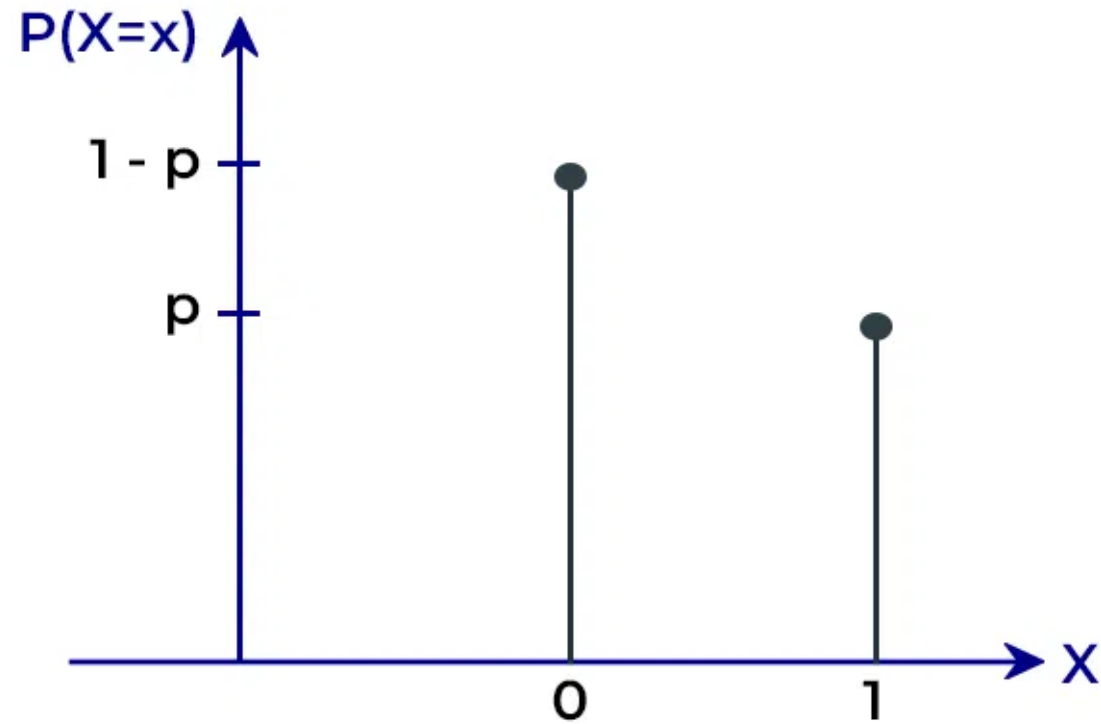
If we sample once from this distribution, what does it mean?

If we sample infinitely many times from this distribution, what do we get?

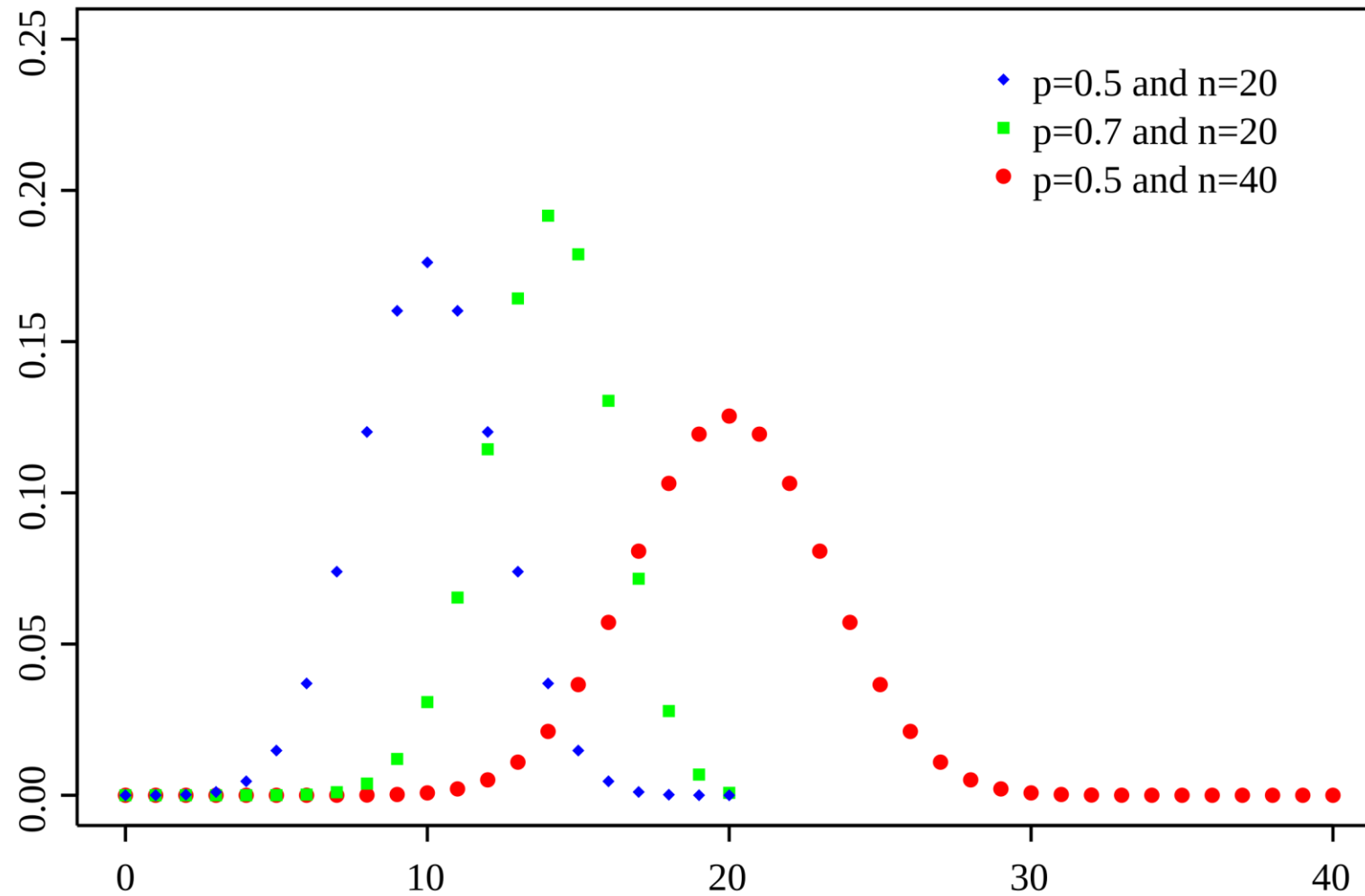
Bernoulli Distributions



$X \sim \text{Bernoulli}(p)$



Binomial Distributions



Model-free Control

Our Plan

Last Lecture

- **Model-free prediction to estimate values in an unknown MDP**
- A brief touch of Deep Reinforcement Learning

This Lecture

- **Model-free control to optimize values in an unknown MDP**

Next Lecture

- Dealing with more dynamic system via function approximation

Model-free Control

A **control** problem for a **model-free** learning

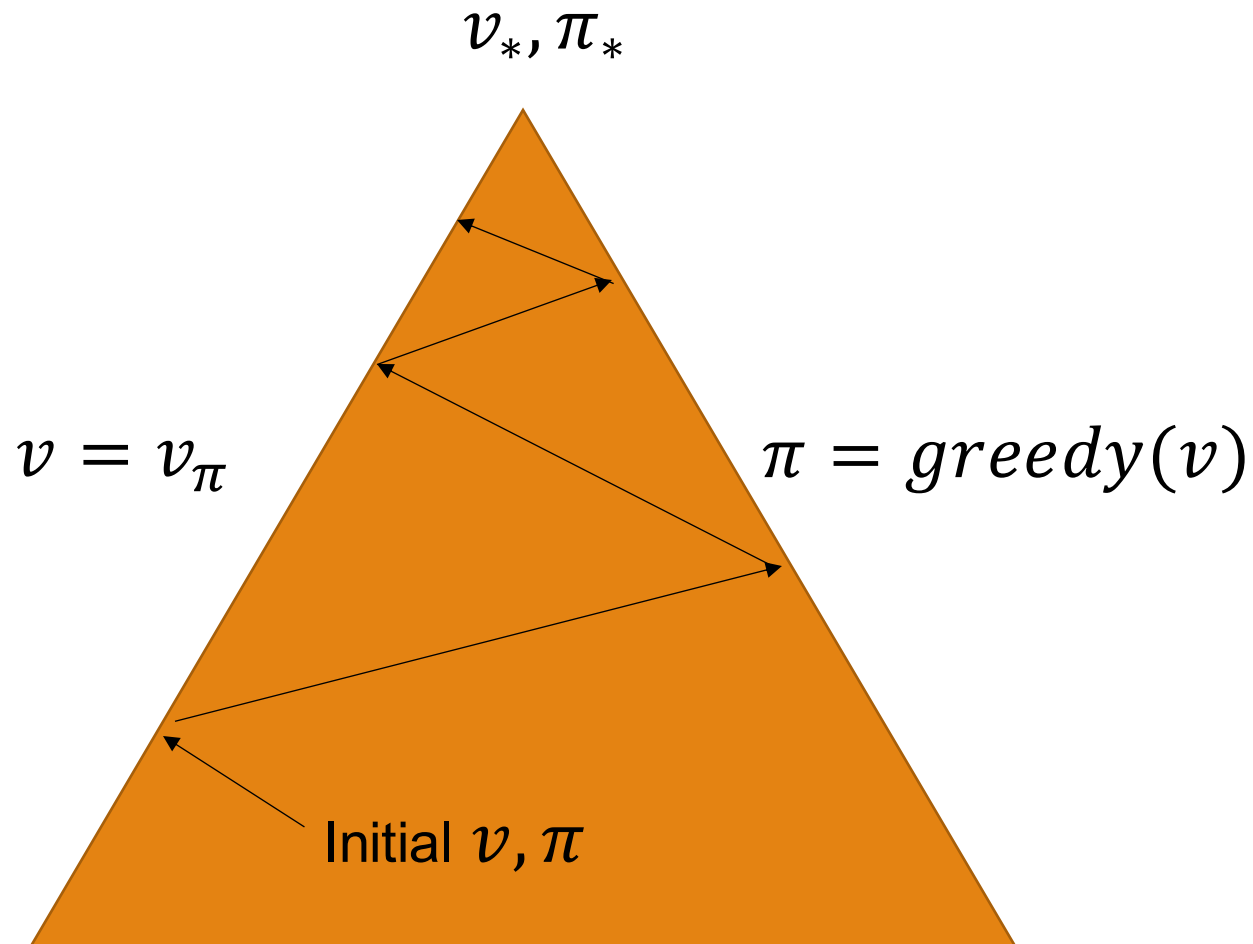
- We do not have a model
- We want to learn the optimal policy

Theme for this lecture:

We learn how to evaluate the policy in the last lecture.

How do we optimize the policy?

Policy Iteration Algorithm Revisited



Starting with initial v, π

Loop do:

Perform policy evaluation,

$$v = v_\pi$$

Perform policy improvement,

$$\pi' \geq \pi$$

Monte Carlo Control

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t-1} R_T$$

MC samples an episode and evaluates its target from state S_t using the equation above. So how do we turn one return into a real policy?

Monte Carlo Control

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t-1} R_T$$

MC samples an episode and evaluates its target from state S_t using the equation above. So how do we turn one return into a real policy?

Answer:

$$\mathbb{E}[G_t] = v_\pi$$

If we sample many times and average the return, we will get a true value given a policy. Why?

Monte Carlo Control

Usually, to find the optimal policy, we can “greedify” the value function.

$$\pi'(s) = \operatorname{argmax}_a \mathbb{E}[R_{t+1} + \gamma v(S_{t+1}) | S_t = s, A_t = a]$$

But we cannot use the equation above, because we cannot find the action that gives the maximum expected value. **In short, we do not know the model.**

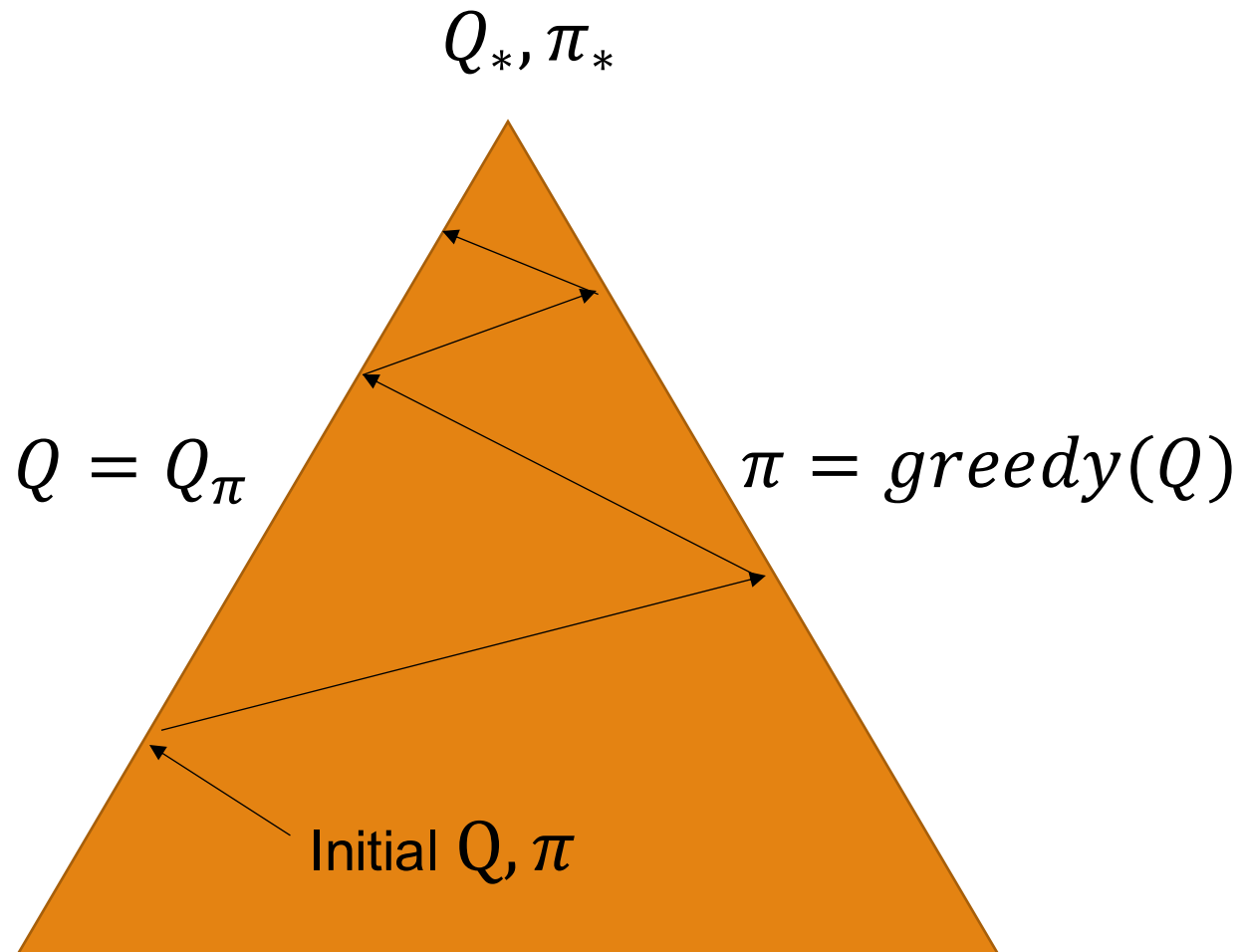
Why do we need to know the model to solve the above equation?

Monte Carlo Control

Instead, we can greedify over $q(s, a)$. This one is model-free.

$$\pi'(s) = \operatorname{argmax}_a q(s, a)$$

Monte Carlo Control



Perform policy evaluation

- Using Monte Carlo policy evaluation

$$q \approx q_\pi$$

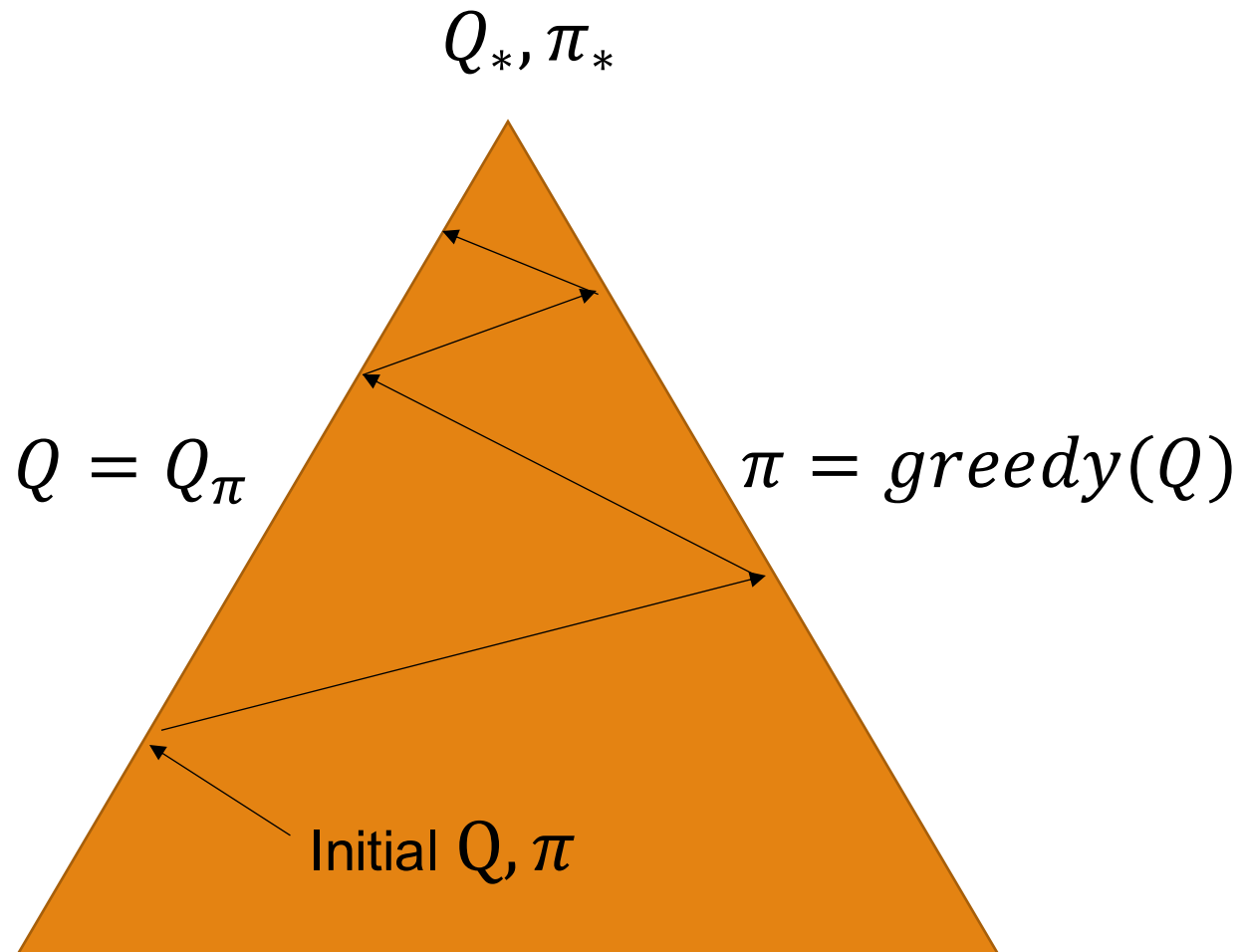
Perform policy improvement

- Using greedy policy improvement

$$\pi' \geq \pi$$

What is the problem here?

Monte Carlo Control



Perform policy evaluation

- Using Monte Carlo policy evaluation

$$q \approx q_\pi$$

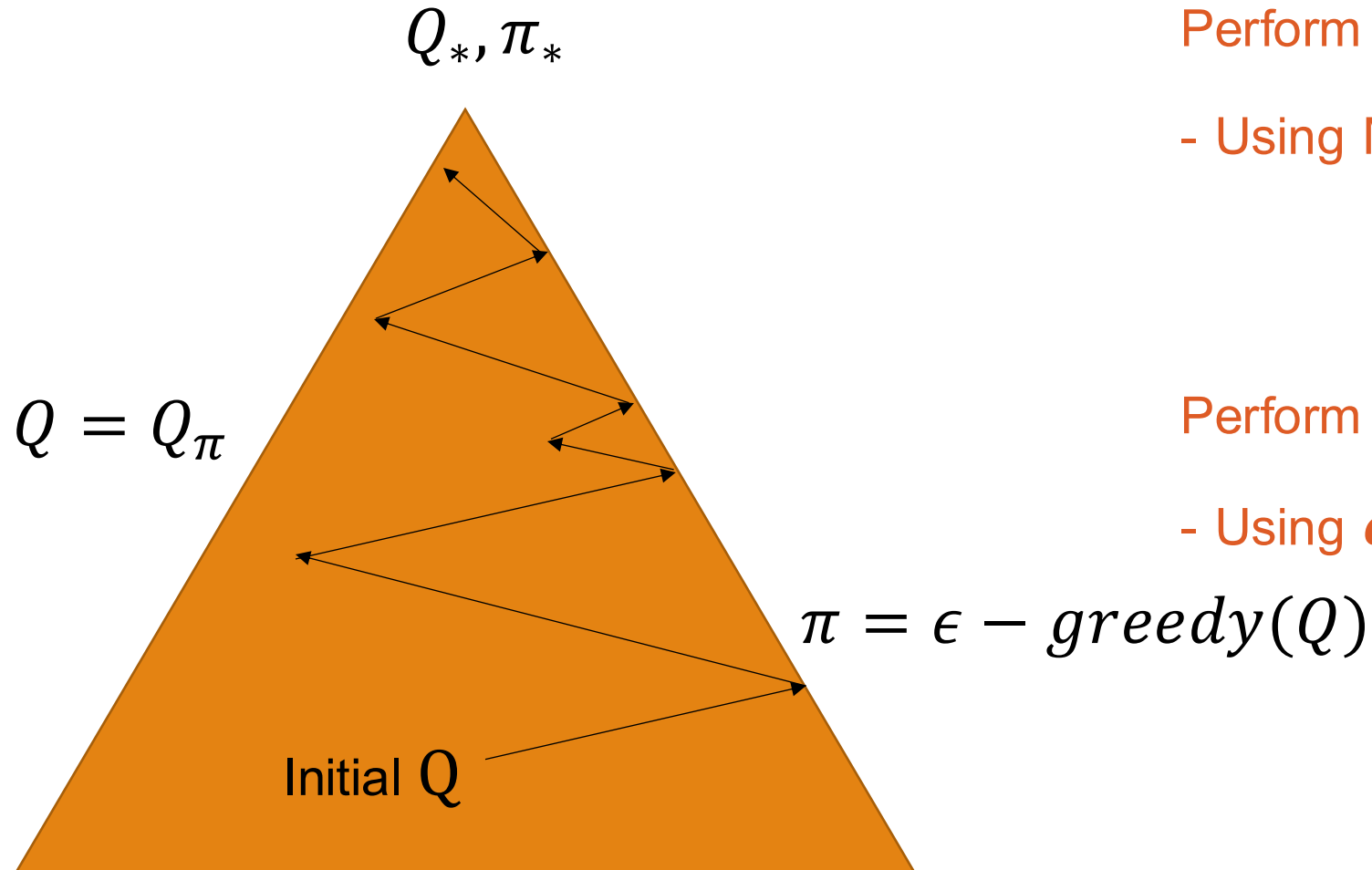
Perform policy improvement

- Using greedy policy improvement

$$\pi' \geq \pi$$

Since we greedify the action value, we have no exploration at all.

Monte Carlo Control



Perform policy evaluation

- Using Monte Carlo policy evaluation
 $q \approx q_\pi$

Perform policy improvement

- Using **ϵ -greedy** policy improvement

GLIE – Greedy in the Limit with Infinite Exploration

This method can be used to find the optimal policy

- Given infinite exploration, all state-action are explored infinitely many times.

$$\lim_{k \rightarrow \infty} N_k(s, a) = \infty$$

- The policy converges to a greedy policy

$$\lim_{k \rightarrow \infty} \pi_k(s, a) = I(a = \operatorname{argmax}_{a'} q_k(s, a'))$$

Example:

- ϵ -greedy with $\epsilon = \frac{1}{k}$

GLIE Monte Carlo Control

Sample k^{th} episode, e_k , using $\pi = \{S_1, A_1, R_2, \dots, S_T\} \sim \pi$

For each S_t and A_t in e_k :

$$N(S_t, A_t) \leftarrow N(S_t, A_t) + 1$$

$$q(S_t, A_t) \leftarrow q(S_t, A_t) + \frac{1}{N(S_t, A_t)} (G_t - q(S_t, A_t))$$

Update the exploration rate and improve policy

$$\epsilon \leftarrow \frac{1}{k}$$

$$\pi \leftarrow \epsilon - greedy(q)$$

Temporal Difference Control

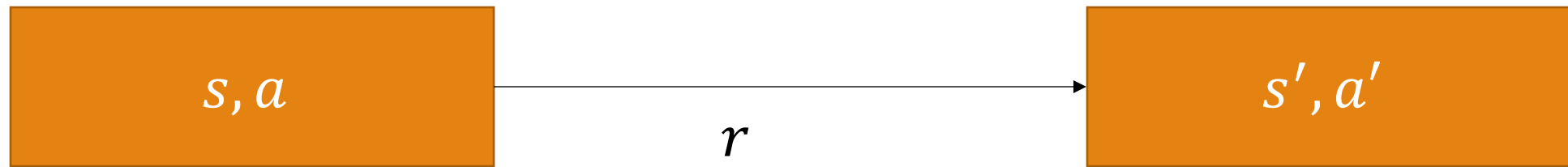
TD learning has many advantages over MC

- Learn from incomplete sequences
- Lower variance
- Online learning

How about we apply the idea to TD instead of MC?

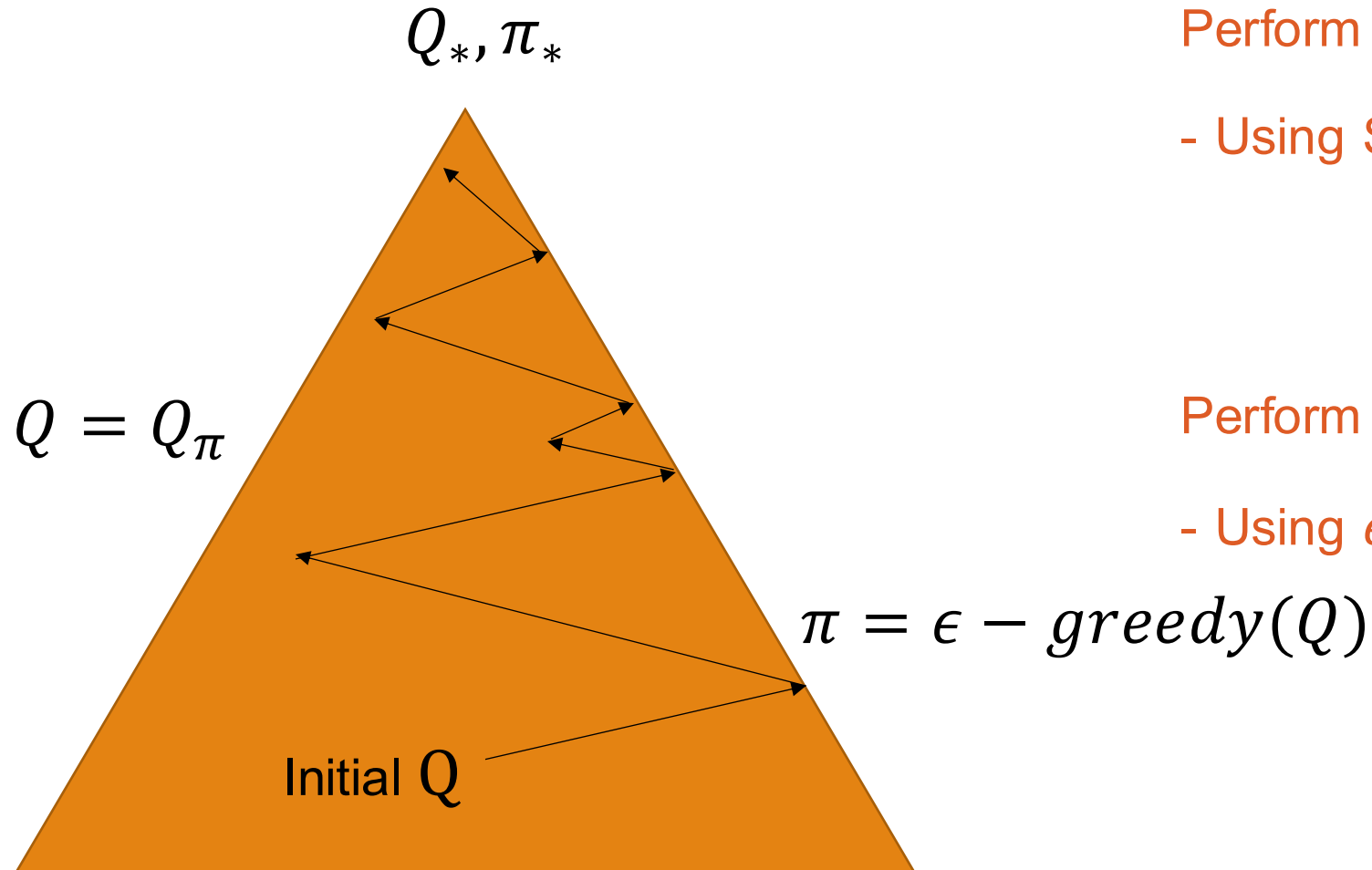
SARSA

SARSA is a model-free reinforcement learning algorithm that learns from **State**, **Action**, **Reward**, **State**, and **Action**



$$q(s, a) \leftarrow q(s, a) + \alpha(r + \gamma q(s', a') - q(s, a))$$

SARSA



Perform policy evaluation

- Using SARSA

$$q \approx q_\pi$$

Perform policy improvement

- Using ϵ -greedy policy improvement

Initialize $Q(s, a)$ arbitrarily

Repeat (for each episode):

Initialize s

Choose a from s using policy derived from Q (e.g., ϵ -greedy)

Repeat (for each step of episode):

Take action a , observe r, s'

Choose a' from s' using policy derived from Q (e.g., ϵ -greedy)

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$$

$s \leftarrow s'; a \leftarrow a';$

until s is terminal

On-Policy and Off-policy Learning

As of now, we let the agent learn from its own experience. Can we do better? Can the agent learn faster? How?

On-Policy and Off-policy Learning

On-policy learning

- Learn from actually performing
- Learn a policy π from samples from π

Off-policy learning

- Learn from other sources
- Learn a policy π from samples from b

Model-based Updates by DP

Policy Evaluation

$$v_{k+1}(s) = \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) | S_t = s, A_t \sim \pi(s)]$$

Value Iteration

$$v_{k+1}(s) = \max_a \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) | S_t = s, A_t = a]$$

Policy Evaluation

$$q_{k+1}(s, a) = \mathbb{E}[R_{t+1} + \gamma q_k(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]$$

Value Iteration

$$q_{k+1}(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_{a'} q_k(S_{t+1}, a') \mid S_t = s, A_t = a \right]$$

Model-free Updates

TD

$$v_{t+1}(S_t) = v_t(S_t) + \alpha_t(R_{t+1} + \gamma v_t(S_{t+1}) - v_t(S_t))$$

SARSA

$$q_{t+1}(S_t, A_t) = q_t(S_t, A_t) + \alpha_t(R_{t+1} + \gamma q_t(S_{t+1}, A_{t+1}) - q_t(S_t, A_t))$$

Q-learning

$$q_{t+1}(S_t, A_t) = q_t(S_t, A_t) + \alpha_t(R_{t+1} + \gamma \max_{a'} q_t(S_{t+1}, a') - q_t(S_t, A_t))$$

Off-policy Learning

Another way of learning. The agent learns from a behavior policy $b(s, a)$ to learn the target policy $\pi(s, a)$

$$\{S_1, A_1, R_2, \dots, S_T\} \sim b$$

For example,

- Learning from others' behavior (human, robot,...)
- Learning from old policies
- Learning from what-if scenarios

Q-learning

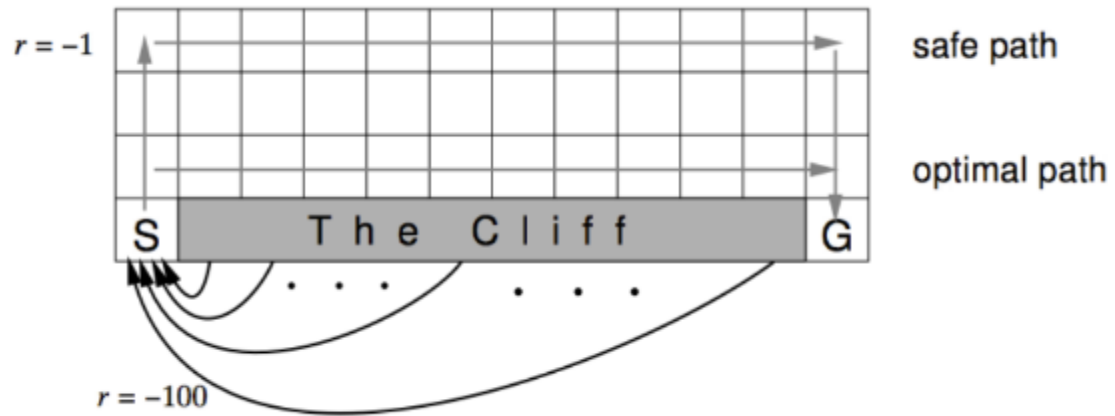
Q-learning is an off-policy, model-free reinforcement learning

$$q_{t+1}(S_t, A_t) = q_t(S_t, A_t) + \alpha_t(R_{t+1} + \gamma \max_{a'} q_t(S_{t+1}, a') - q_t(S_t, A_t))$$

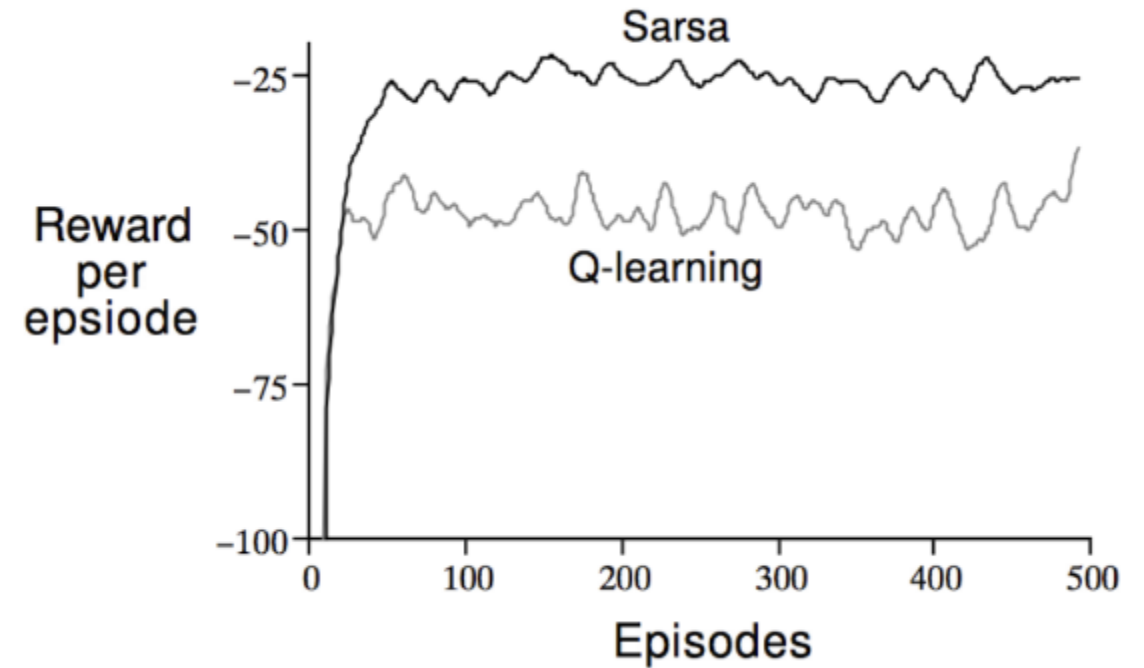
Q-learning learns from the value of the greedy policy, thus an off-policy learning.

- It converges to the optimal action-value function when we explore all state-action infinitely many times
- No greedy behavior needed

SARSA vs Q-learning



Safe path by SARSA, and optimal path by Q-learning



Reward from SARSA and Q-learning

Q-learning Issue

Q-learning uses the same value to select actions and evaluate its value.

$$\max_a q_t(S_{t+1}, a) = q_t(S_{t+1}, \operatorname{argmax}_a q_t(S_{t+1}, a))$$

However, these values are approximated.

So it will **select overestimated values more** and **select underestimate values less**.

Double Q-learning

Since

$$\max_a q_t(S_{t+1}, a) = q_t(S_{t+1}, \operatorname{argmax}_a q_t(S_{t+1}, a))$$

Q-learning will use the same values to select action, and evaluate the value

$$R_{t+1} + \gamma \max_{a'} q_t(S_{t+1}, a') = R_{t+1} + \gamma q_t(S_{t+1}, \operatorname{argmax}_a q_t(S_{t+1}, a))$$

And this will result in the issue in the last slide. To solve this problem, we can introduce another policy to split action selection from evaluation.

Double Q-learning

Double Q-learning will

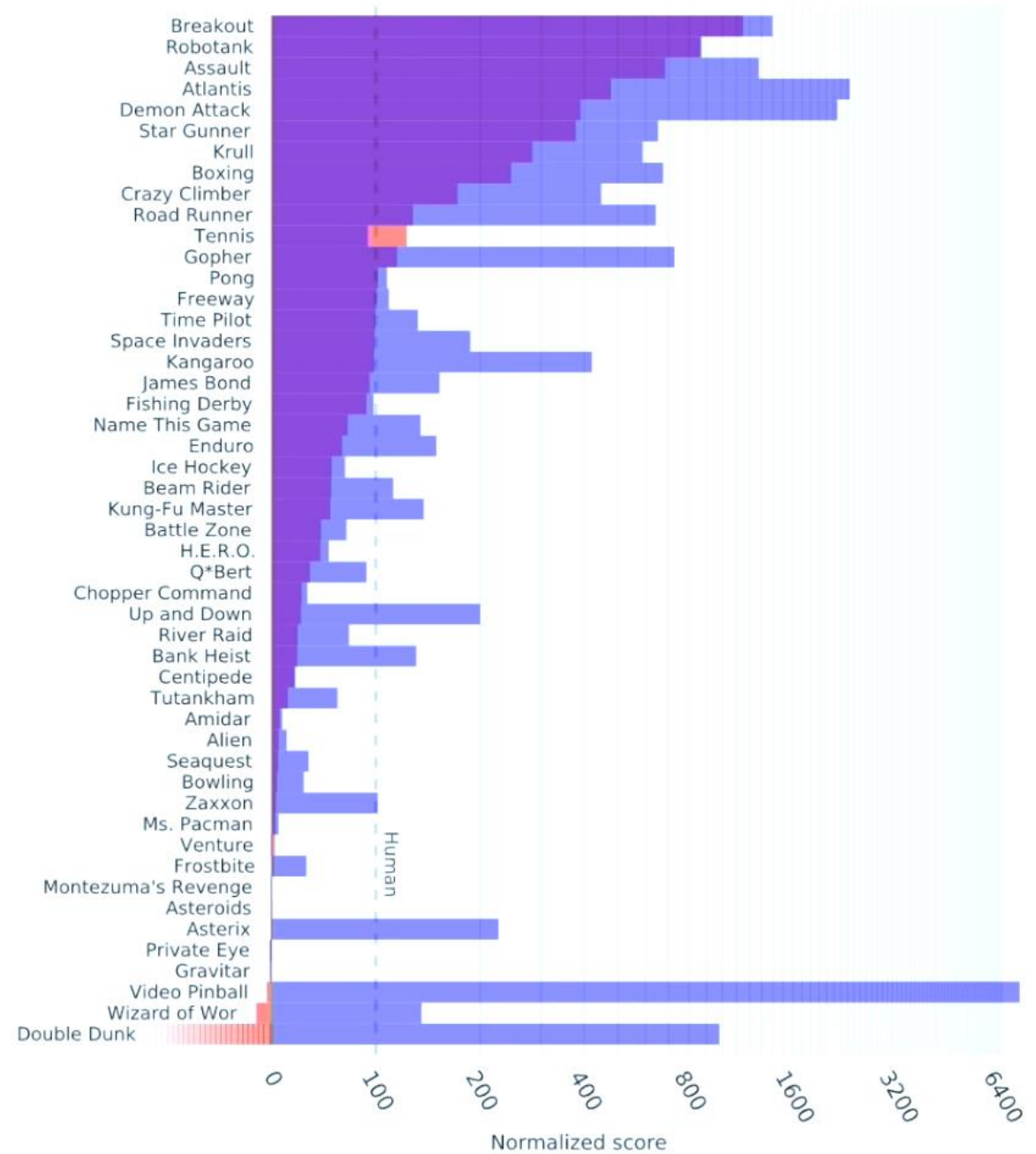
- store two q functions: q and q'
- update one q function for each iteration.

$$R_{t+1} + \gamma q'_t(S_{t+1}, \operatorname{argmax}_a q_t(S_{t+1}, a))$$

$$R_{t+1} + \gamma q_t(S_{t+1}, \operatorname{argmax}_a q'_t(S_{t+1}, a))$$

DQN

Double DQN



Generalized Q-learning

We can also learn from a behavior policy $b(s, a)$ while using $\pi(s, a)$

We modify Q-learning to choose an action from behavior policy $A_{t+1} \sim b(S_{t+1}, \cdot)$

$$q(S_t, A_t) \leftarrow q(S_t, A_t) + \alpha_t \left(R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) q(S_{t+1}, a) - q(S_t, A_t) \right)$$

When $b = \pi$, we can call this equation Expected SARSA

Off-policy Q-Learning Control

To control, we want to optimize both policies.

For the target policy π , we greedify the action on $q(s, a)$

$$\pi(S_{t+1}) = \operatorname{argmax}_{a'} q(S_{t+1}, a')$$

For the behavior policy b , we can use ϵ -greedy on $q(s, a)$

Thus, Q-Learning target is simplified as:

$$R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) q(S_{t+1}, a) = R_{t+1} + \gamma \max_a q(S_{t+1}, a)$$